Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Phase 1 – Data model

1. Application Domain

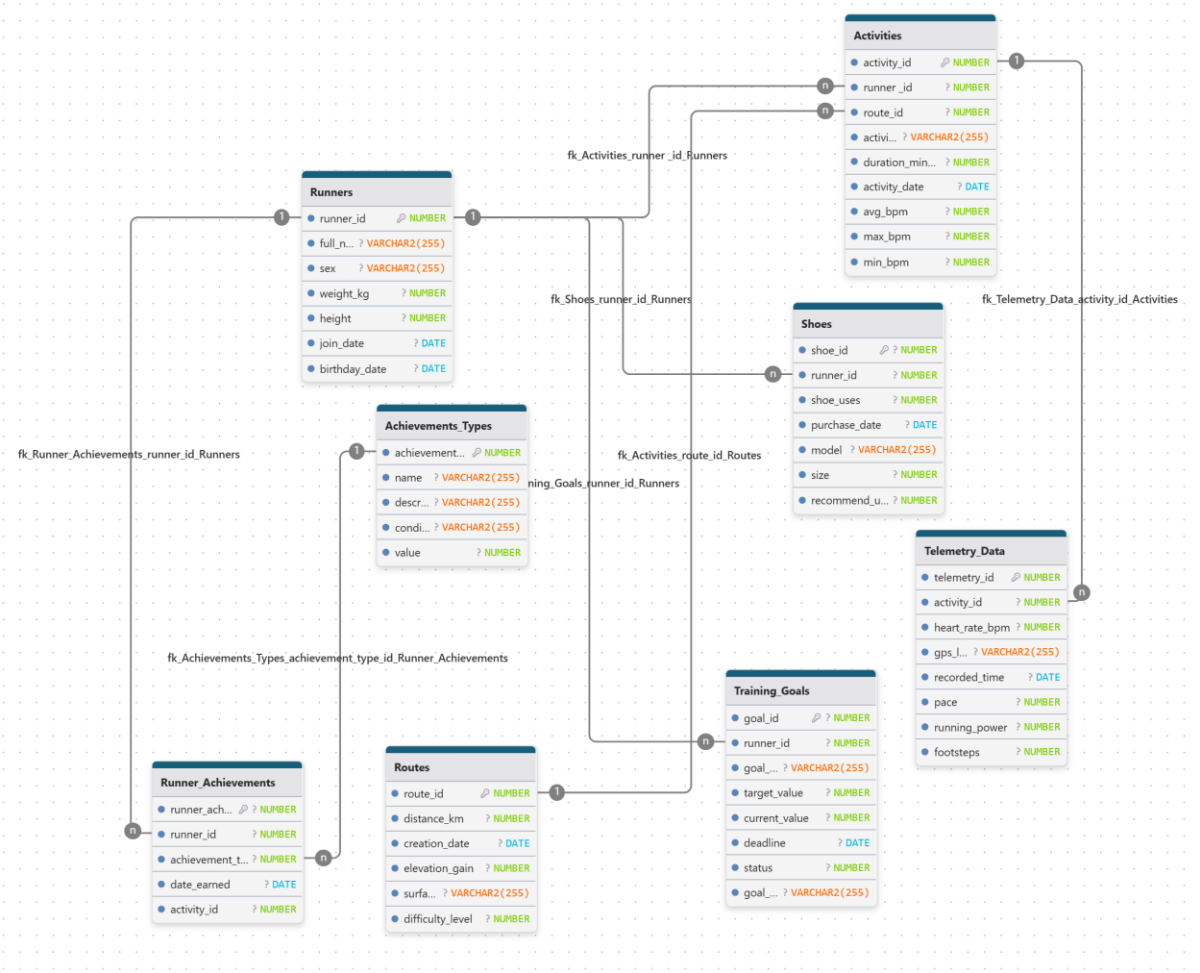
Topic: Running Tracking System

Description: A database system for a running application that records users' profiles, their overall workout sessions, and telemetry data such as heart rate. It also tracks the lifecycle of the runners' shoes, routes, and earned achievements.

2. Data Model Diagram & Specification

Entity-Relationship Overview (Associations):

- One **Runner** can have many **Shoes**, **Activities**, **Training_Goals**, and **Runner_Achievements** (1-to-Many).
- One **Route** can be used in many **Activities** (1-to-Many).
- One **Activity** contains many points of **Telemetry_Data** (1-to-Many).
- One **Activity** can result in many **Runner_Achievements** (1-to-Many).
- One **Achievement_Type** can be awarded as many **Runner_Achievements** (1-to-Many).



Assignment: Phase 2 - Workload

1. Workload Description

This workload simulates the real-world operational demands of the Running Tracking System. It consists of complex transactions designed to guarantee observable execution times for performance tuning. The workload is divided into two categories: complex querying operations that simulate analytical reporting across multiple joined relations, and data-changing operations that handle heavy telemetry data usage and background data recalculations.

2. Query operations.

- **First query operation: Analysis of Elite Male Runners**
    - **Description:** This query identifies male runners who joined the system within the last 100 days , weigh less than 90 kg, and have covered a total distance in 2026 that exceeds the average for this specific demographic. The analysis focuses on performance on challenging routes (difficulty level 3 or higher) to isolate high-intensity training data.
    - **Database action:** The query joins the **Runners**, **Activities**, and **Routes** tables. It applies a strict date filter for activities performed in 2026 and filters runners based on specific attributes: sex ('Male'), weight (under 90 kg), and a recent join date (within the last 100 days). Only routes with a technical difficulty level of 3 or more are included. A subquery is used to calculate the average total distance specifically for this filtered group of male runners. The final output displays each runner's ID and full name along with their total distance in kilometers, sorted in descending order.

```
SELECT
   r.runner_id, r.full_name,
   SUM(ro.distance_km) AS total_distance_km
FROM Runners r
JOIN Activities a ON r.runner_id = a.runner_id
JOIN Routes ro ON a.route_id = ro.route_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
 -- Dodajemy nowe filtracje:
 AND r.sex = 'Male'
 AND r.weight_kg < 90
 AND r.join_date > (SYSDATE - 100)
 AND ro.difficulty_level >= 3
```

```
GROUP BY
  r.runner_id,
  r.full_name
HAVING SUM(ro.distance_km) > (
  SELECT AVG(total_distance)
  FROM (
    SELECT SUM(ro2.distance_km) AS total_distance
    FROM Activities a2
    JOIN Routes ro2 ON a2.route_id = ro2.route_id
    JOIN Runners r2 ON a2.runner_id = r2.runner_id
    WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
     AND r2.sex = 'Male'
     AND r2.weight_kg < 90
    GROUP BY a2.runner_id
  )
)
ORDER BY total_distance_km DESC;
```

- **Second query operation: Top 10 High-Altitude Performers**
  - **Description:** This query identifies the top 10 runners taller than 175 cm who utilized specific equipment—specifically Nike brand shoes—and completed runs on routes with recorded elevation gains in 2026. It highlights high-performing athletes whose total distance exceeds the yearly global average while simultaneously tracking their total number of earned achievements.
  - **Database action:** The database executes a complex multi-table join involving **Runners**, **Activities**, **Routes**, **Shoes**, and a left join with **Runner_Achievements**. Several precise filters are applied: the activity year is set to 2026, runner height must be over 175 cm, the shoe model must match the 'Nike%' pattern, and the route must have an elevation gain greater than 10 meters. A subquery calculates the average total distance covered by all runners in 2026 to serve as a benchmark. Finally, the data is grouped by runner identity to aggregate distances and achievements, returning only the top 10 records sorted by total distance in descending order.
  -

```
SELECT
```

```
  r.runner_id,
  r.full_name,
  SUM(ro.distance_km) AS total_distance_km,
  COUNT(DISTINCT ra.runner_achievement_id) AS total_achievements
FROM Runners r
JOIN Activities a
  ON r.runner_id = a.runner_id
JOIN Routes ro
  ON a.route_id = ro.route_id
LEFT JOIN Runner_Achievements ra
  ON r.runner_id = ra.runner_id
LEFT JOIN Shoes s
  ON a.runner_id = s.runner_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
 AND r.height > 175
 AND s.model LIKE 'Nike%'
 AND ro.elevation_gain > 10
GROUP BY
  r.runner_id,
  r.full_name
HAVING SUM(ro.distance_km) > (
  SELECT AVG(total_distance)
  FROM (
    SELECT SUM(ro2.distance_km) AS total_distance
    FROM Activities a2
    JOIN Routes ro2
      ON a2.route_id = ro2.route_id
    WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
    GROUP BY a2.runner_id
  )
)
ORDER BY total_distance_km DESC
FETCH FIRST 10 ROWS ONLY;
```

- **Third query operation: Heart Rate Efficiency in Heavyweight Categories**
  - **Description:** This query identifies "efficient" runners within the 65–95 kg weight ranges whose average heart rate during 2026 was lower than the collective average for their specific weight class. To ensure statistical significance and focus on high-intensity training, the query only includes runners who completed at least 5 activities characterized by a fast pace and a duration exceeding 20 minutes.

- **Database action:** The database performs a join across the **Runners**, **Activities**, and **Telemetry_Data** tables. It filters for records from the year 2026, targeting runners weighing between 65 and 95 kg. Additional filters are applied to the telemetry and activity data, specifically requiring a pace faster than 6.0 and a session duration of more than 20 minutes. A subquery calculates the overall average heart rate for all runners within this weight category who met the minimum requirement of 5 activities. The final result set includes only those runners whose personal average heart rate is below the group average, with results sorted from the lowest (most efficient) to the highest average heart rate.

```
SELECT
  r.runner_id,
  r.full_name,
  ROUND(AVG(t.heart_rate_bpm), 2) AS avg_heart_rate,
  COUNT(DISTINCT a.activity_id) AS total_activities
FROM Runners r
JOIN Activities a
  ON r.runner_id = a.runner_id
JOIN Telemetry_Data t
  ON a.activity_id = t.activity_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
 AND r.weight_kg BETWEEN 65 AND 95
 AND t.pace < 6.0
 AND a.duration_min > 20
GROUP BY
  r.runner_id,
  r.full_name
HAVING COUNT(DISTINCT a.activity_id) >= 5
  AND AVG(t.heart_rate_bpm) < (
    SELECT AVG(runner_avg_hr)
    FROM (
      SELECT AVG(t2.heart_rate_bpm) AS runner_avg_hr
      FROM Activities a2
      JOIN Telemetry_Data t2
        ON a2.activity_id = t2.activity_id
      JOIN Runners r2
        ON a2.runner_id = r2.runner_id
      WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
       AND r2.weight_kg BETWEEN 65 AND 95
      GROUP BY a2.runner_id
      HAVING COUNT(DISTINCT a2.activity_id) >= 5
    )
  )
ORDER BY avg_heart_rate ASC;
```

3. Changing operations

→ The **INSERT** operation: "Syncing a Smartwatch Workout"

- **Description:** This operation represents the first step of the data synchronization process after a user finishes a run. It focuses on saving the overall workout summary into the **Activities** table. This record serves as the primary reference for the session, capturing aggregate performance data before the system proceeds to log detailed telemetry points.
- **Database Action:** The transaction executes an `INSERT` statement to add a new summary row to the **Activities** table. To maintain unique identification without an automatic counter, the script uses a subquery to calculate the next `activity_id` by finding the current maximum value and incrementing it by one.

The inserted record consists of:

- **User and Route Context**: Identification of the runner (e.g., `runner_id = 501`) and the specific route used (`route_id = 10`).
- **Session Classification**: Categorizing the workout type (e.g., 'Easy Run') and recording the date (April 10, 2026).
- **Performance Metrics**: The total duration of 47 minutes and comprehensive heart rate data, including the average (`avg_bpm`), maximum (`max_bpm`), and minimum (`min_bpm`) values captured during the run.

```
-- 1. Wstawienie nagłówka aktywności (Summary Row)

INSERT INTO Activities (

  activity_id,

  runner_id,

  route_id,

  activity_type,

  duration_min,

  activity_date,

  avg_bpm,
```

```
  max_bpm,
  min_bpm
)
VALUES (
  (SELECT MAX(activity_id) + 1 FROM Activities),
  501,
  10,
  'Easy Run',
  47,
  DATE '2026-04-10',
  148,
  162,
  129
);
```

→ The **UPDATE** operation: "Nightly Training Goal Recalculation"

- **Description:** Every night, the application executes a background job to monitor and update the progress of active training goals for each runner. This operation ensures that users receive real-time feedback on their achievements by synchronizing their recent activity data with their defined targets. To optimize performance, the update is restricted to "Active" goals belonging to users who have interacted with the system within the last week.
- **Database Action:** This transaction performs a complex UPDATE on the **Training_Goals** table. It employs several advanced SQL techniques:
  - **Dynamic Progress Tracking**: The `current_value` column is recalculated using a correlated subquery that sums the `distance_km` from the **Routes** and **Activities** tables. The subquery is filtered to only

include activities that occurred between the goal's specific `start_date` and its `deadline`.

- **Conditional Status Logic**: A `CASE` expression is used to automatically evaluate if a goal has been reached. If the total distance covered meets or exceeds the `target_value`, the status is updated to '1' (Completed); otherwise, it remains '0' (In Progress).
- **Targeted Processing**: The operation includes a `WHERE` clause that limits the update to goals currently 'In Progress'. Furthermore, a subquery on the **Runners** table filters for active participants who have logged in within the last 7 days (`SYSDATE - 7`), preventing unnecessary calculations for inactive accounts.

```
UPDATE Training_Goals tg

SET tg.current_value = ( SELECT SUM(ro.distance_km)

FROM Activities a

JOIN Routes ro ON a.route_id = ro.route_id

WHERE a.runner_id = tg.runner_id

AND a.activity_date >= tg.start_date

AND a.activity_date <= tg.deadline ),

tg.status = CASE

WHEN (SELECT SUM(ro2.distance_km) FROM Activities a2 JOIN Routes ro2 ON a2.route_id = ro2.route_id WHERE a2.runner_id = tg.runner_id) >= tg.target_value

THEN '1' -- Completed

ELSE '0' -- In Progress

END

WHERE tg.status = '0'

AND tg.runner_id IN ( SELECT runner_id FROM Runners WHERE last_login > SYSDATE - 7);
```

→ The **DELETE** Transaction: "Account Deletion"

- **Description:** This operation handles a runner's request to permanently remove their profile and all associated fitness data from the system. Due to

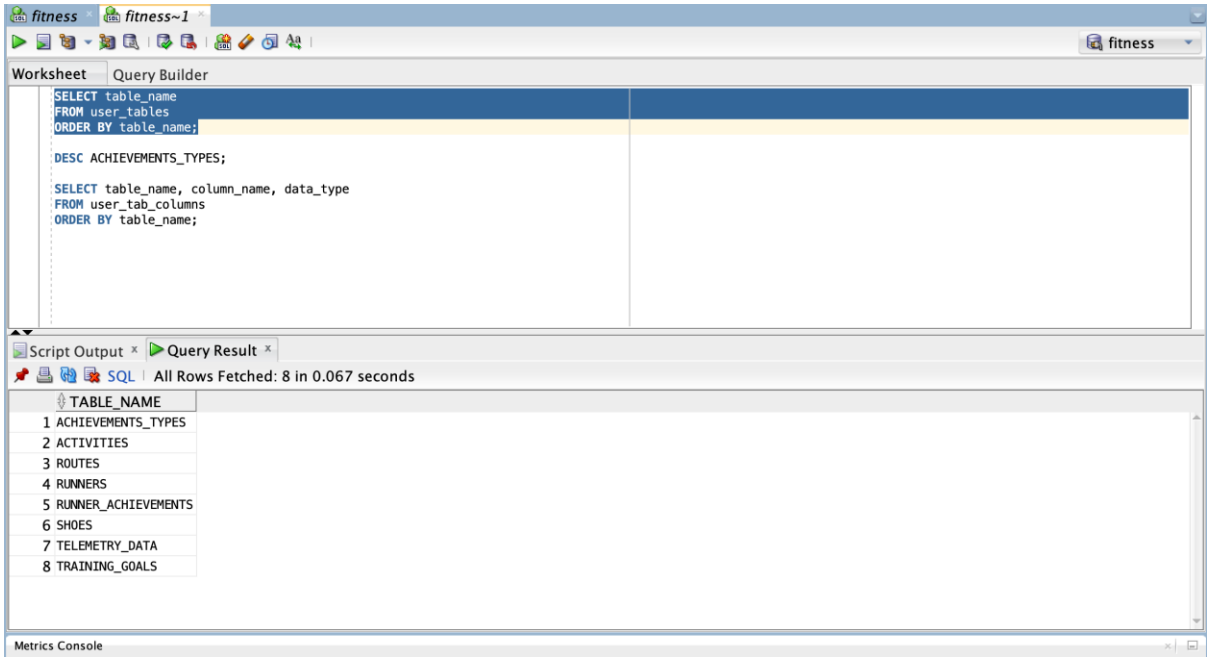
strictly enforced **Referential Integrity** within the database schema, the system cannot delete a user profile while dependent records exist. This transaction ensures a clean and complete data wipe by following a specific hierarchical order, preventing orphaned records and maintaining database consistency.

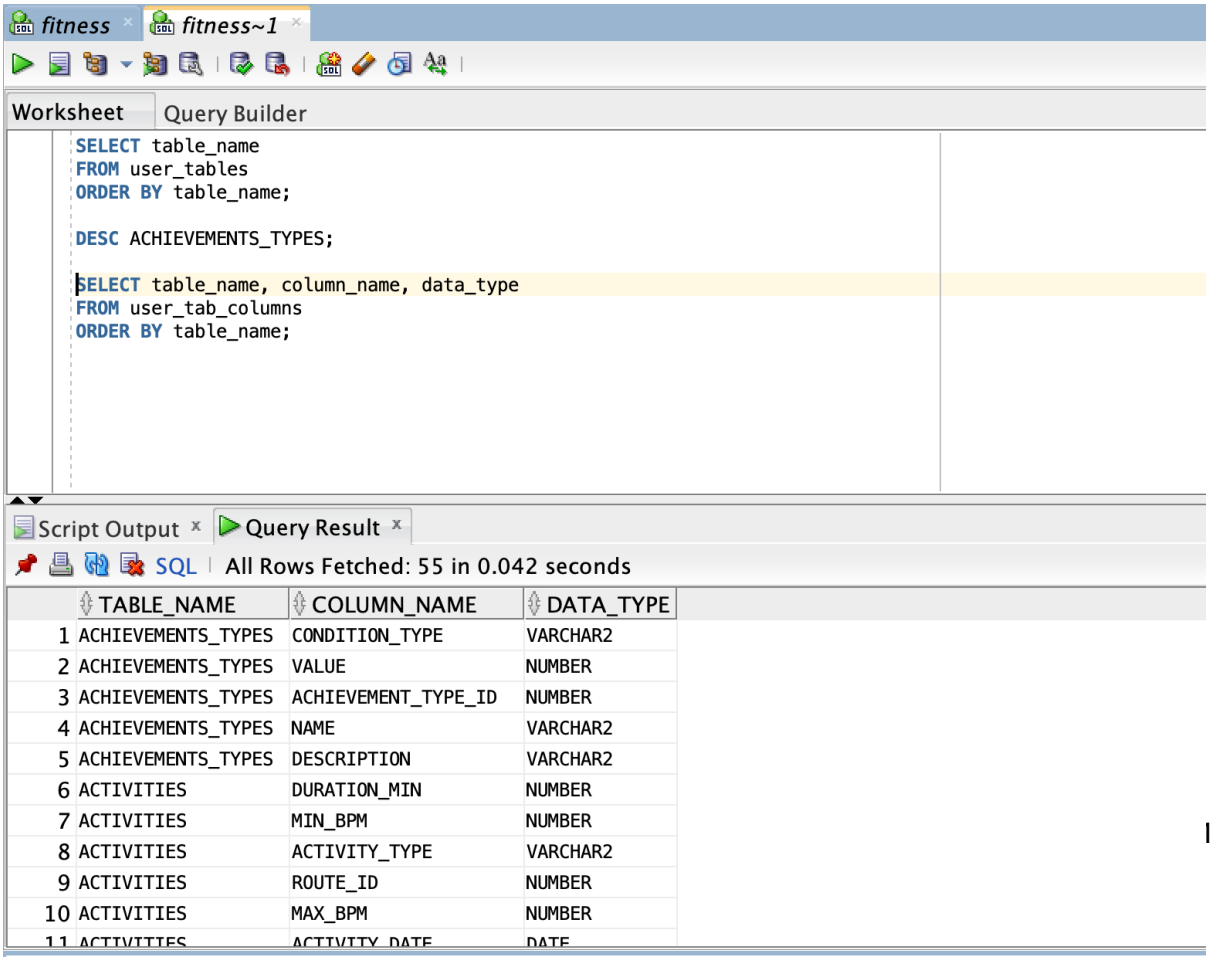
- **Database Action:** This transaction consists of a multi-step sequence of `DELETE` statements, culminating in a `COMMIT` to finalize the changes. The operation is executed in the following mandatory order:
  - **Telemetry and Achievements Removal**: The process begins by wiping high-volume data. It first deletes thousands of rows from the **Telemetry_Data** and **Runner_Achievements** tables using subqueries to identify activities linked to the specific runner (e.g., `runner_id = 501`).
  - **Secondary Data Cleanup**: The transaction then proceeds to remove personal progress data, including records from the **Training_Goals**, **Runner_Achievements** (direct link), and **Shoes** tables.
  - **Activity Deletion**: Once all dependent records are removed, the system deletes the core records from the **Activities** table.
  - **Profile Termination**: The final step is the deletion of the user from the **Runners** table. This statement includes a safety filter ensuring the account is not brand new (`join_date < SYSDATE - 1`) to prevent accidental deletions of recently created profiles.

```
-- Wielostopniowe usuwanie kaskadowe (Referential Integrity

DELETE FROM Telemetry_Data

WHERE activity_id IN (

  SELECT activity_id

  FROM Activities

  WHERE runner_id = 501

);


DELETE FROM Runner_Achievements

WHERE activity_id IN (
```

```
   SELECT activity_id
   FROM Activities
   WHERE runner_id = 501
);

DELETE FROM Runner_Achievements
WHERE runner_id = 501;

DELETE FROM Training_Goals
WHERE runner_id = 501;

DELETE FROM Shoes
WHERE runner_id = 501;

DELETE FROM Activities
WHERE runner_id = 501;

DELETE FROM Runners
WHERE runner_id = 501
  AND join_date < SYSDATE - 1;

COMMIT;
```

The database was created using Oracle Database 21c Express Edition, following the professor's recommendation on eportal. A dedicated user schema was created, and the database structure was implemented using SQL DDL statements.





Data + Database workload

Data generation:

The data was generated using a custom Python-based data generator. The implementation was supported by ChatGPT, which was used to design and refine the generation logic to achieve the desired scale and realistic relationships between entities. The script ensures consistency across tables and simulates real-world fitness tracking data, including activities, routes, and telemetry records.

Table name	Number of rows
RUNNERS	1200
ROUTES	250
SHOES	2400
ACTIVITIES	36000
TELEMETRY_DATA	1079798
TRAINING_GOALS	1800

ACHIEVEMENTS_TYPES	12
RUNNER_ACHIEVEMENTS	5000

Volumetry description:

The database was populated with a large sample dataset reflecting realistic dependencies between the entities of the running tracking system. The largest table is Telemetry_Data, which contains over 1 million rows, because each activity stores multiple telemetry measurements. The Activities table contains 36,000 rows, while the remaining tables were scaled proportionally to preserve realistic relations between runners, routes, shoes, goals, and achievements.